

LAPORAN MODUL 9 PRAKTIKUM PARADIGMA PEMGROGRAMAN



Disusun oleh :

Nama : Irvan Cahya Nugraha
NIM : 245410034
Kelas : Informatika 1

**PROGRAM STUDI INFORMATIKA
PROGRAM SARJANA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS TEKNOLOGI DIGITAL INDONESIA
YOGYAKARTA
2024 / 2025**

MODUL 9

KELAS ABSTRAK

A. TUJUAN PRAKTIKUM

1. Dapat membuat dan menggunakan kelas abstrak

B. DASAR TEORI

Abstraksi mengacu pada kemampuan untuk membuat kelas abstrak di OOP. Sebuah kelas abstrak adalah salah satu yang tidak bisa dipakai. Semua fungsi lain dari kelas masih ada, dan ladang-ladangnya, metode, dan konstruktor semua diakses dengan cara yang sama. Anda hanya tidak bisa membuat sebuah instance dari kelas abstrak.

Jika kelas abstrak dan tidak dapat dipakai, kelas tidak memiliki banyak digunakan kecuali subclass. Ini biasanya bagaimana kelas abstrak terjadi selama tahap desain. Sebuah kelas induk berisi fungsi umum dari kumpulan anak kelas, tapi kelas induk itu sendiri terlalu abstrak untuk digunakan sendiri. Abstrak Kelas: Gunakan kata kunci abstract untuk menyatakan abstract class. Kata kunci muncul dalam deklarasi kelas di suatu tempat sebelum kata kunci kelas

C. PEMBAHASAN LISTING

PRAKTIK

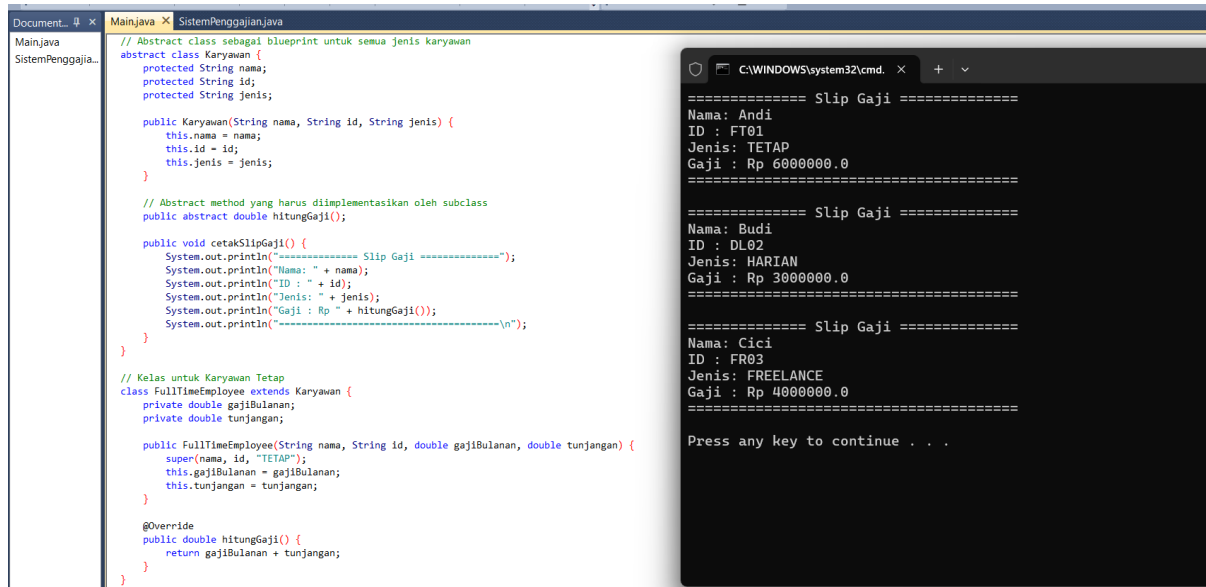
Praktik 1

Pembahasan :

LATIHAN

D. PEMBAHASAN TUGAS

Tugas 1



The image shows a Java IDE on the left and a Windows command prompt on the right. The IDE displays the source code for a payroll system. The code defines an abstract class `Karyawan` with attributes `nama`, `id`, and `jenis`, and methods `hitungGaji()` and `cetakSlipGaji()`. A concrete class `FullTimeEmployee` extends `Karyawan` and implements the `hitungGaji()` method. The command prompt shows the output of the program, displaying the payroll slip for three employees: Andi, Budi, and Cici.

```
Document_ x Mainjava x SistemPenggajian.java
Mainjava
SistemPenggajian...

// Abstract Class sebagai blueprint untuk semua jenis karyawan
abstract class Karyawan {
    protected String nama;
    protected String id;
    protected String jenis;

    public Karyawan(String nama, String id, String jenis) {
        this.nama = nama;
        this.id = id;
        this.jenis = jenis;
    }

    // Abstract method yang harus diimplementasikan oleh subclass
    public abstract double hitungGaji();

    public void cetakSlipGaji() {
        System.out.println("===== Slip Gaji =====");
        System.out.println("Nama: " + nama);
        System.out.println("ID : " + id);
        System.out.println("Jenis: " + jenis);
        System.out.println("Gaji : Rp " + hitungGaji());
        System.out.println("-----\n");
    }
}

// Kelas untuk Karyawan Tetap
class FullTimeEmployee extends Karyawan {
    private double gajiBulanan;
    private double tunjangan;

    public FullTimeEmployee(String nama, String id, double gajiBulanan, double tunjangan) {
        super(nama, id, "TETAP");
        this.gajiBulanan = gajiBulanan;
        this.tunjangan = tunjangan;
    }

    @Override
    public double hitungGaji() {
        return gajiBulanan + tunjangan;
    }
}

===== Slip Gaji =====
Nama: Andi
ID : FT01
Jenis: TETAP
Gaji : Rp 6000000.0
=====

===== Slip Gaji =====
Nama: Budi
ID : DL02
Jenis: HARIAN
Gaji : Rp 3000000.0
=====

===== Slip Gaji =====
Nama: Cici
ID : FR03
Jenis: FREELANCE
Gaji : Rp 4000000.0
=====

Press any key to continue . . .
```

```
Main.java x SistemPenggajian.java

// Kelas untuk Karyawan Harian
class DailyEmployee extends Karyawan {
    private int jumlahHariKerja;
    private double upahHarian;

    public DailyEmployee(String nama, String id, int jumlahHariKerja, double upahHarian) {
        super(nama, id, "HARIAN");
        this.jumlahHariKerja = jumlahHariKerja;
        this.upahHarian = upahHarian;
    }

    @Override
    public double hitungGaji() {
        return jumlahHariKerja * upahHarian;
    }
}

// Kelas untuk Karyawan Freelance
class FreelanceEmployee extends Karyawan {
    private int jumlahJamKerja;
    private double ratePerJam;

    public FreelanceEmployee(String nama, String id, int jumlahJamKerja, double ratePerJam) {
        super(nama, id, "FREELANCE");
        this.jumlahJamKerja = jumlahJamKerja;
        this.ratePerJam = ratePerJam;
    }

    @Override
    public double hitungGaji() {
        return jumlahJamKerja * ratePerJam;
    }
}

// Main class untuk menjalankan program
public class Main {
    public static void main(String[] args) {
        // Membuat array karyawan (tanpa ArrayList)
        Karyawan[] daftarKaryawan = new Karyawan[3];

        // Mengisi array dengan karyawan
        daftarKaryawan[0] = new FullTimeEmployee("Andi", "FT01", 5500000, 500000);

        // Mengisi array dengan karyawan
        daftarKaryawan[0] = new FullTimeEmployee("Andi", "FT01", 5500000, 500000);
        daftarKaryawan[1] = new DailyEmployee("Budi", "DL02", 20, 150000);
        daftarKaryawan[2] = new FreelanceEmployee("Cici", "FR03", 160, 25000);

        // Mencetak slip gaji untuk semua karyawan
        for (int i = 0; i < daftarKaryawan.length; i++) {
            daftarKaryawan[i].cetakSlipGaji();
        }
    }
}
```

Pembahasan : Program di atas adalah program Java yang mengimplementasikan abstract class Karyawan sebagai blueprint. Abstract class ini memiliki variabel nama, id, jenis dan abstract method hitungGaji(). Tiga subclass yaitu FullTimeEmployee, DailyEmployee, dan FreelanceEmployee mewarisi class Karyawan dan mengimplementasikan method hitungGaji() dengan rumus berbeda-beda sesuai jenis karyawan. Di class Main, array daftarKaryawan diisi dengan objek dari ketiga subclass, kemudian method cetakSlipGaji() dipanggil untuk setiap objek, menghasilkan perhitungan gaji yang berbeda secara polimorfik.

E. KESIMPULAN

Praktikum ini berhasil mendemonstrasikan penerapan abstract class dan pewarisan dalam pemrograman Java. Abstract class **Karyawan** berfungsi efektif sebagai blueprint yang memaksa subclass untuk mengimplementasikan method **hitungGaji()**. Konsep polimorfisme terbukti bekerja dengan baik dimana method **cetakSlipGaji()** dapat memanggil implementasi **hitungGaji()** yang berbeda-beda sesuai tipe objeknya. Struktur program ini menunjukkan keunggulan OOP dalam mengelola sistem dengan entitas serupa namun memiliki perilaku perhitungan yang berbeda.

F. LAMPIRAN